Title: Experimental Analysis, Optimization, and Design Patterns in Unix Shell Scripts for System-Level Automation

Sidrit RRUSTEMI, Viola ZENELI, Artemisa HASALAMI

Department of Computer Engineering, EPOKA University, Tirana, Albania

Abstract

Shell scripting is an essential component of Unix-based operating systems and is widely used for automation, system administration, text processing, and file management tasks. Despite its extensive use, shell script development often prioritizes functionality over performance, resulting in implementations that may contain unnecessary overhead. This paper presents an experimental analysis of ten representative shell scripts commonly used in Unix environments. Each script was evaluated in terms of functionality, computational complexity, execution behavior, and resource utilization.

The study employed standard Linux performance analysis tools, including time, strace, and top, to measure execution time, memory consumption, and system activity. In addition, optimized versions of the selected scripts were developed and compared with their original implementations. The optimization process focused on reducing external command usage, minimizing unnecessary pipelines, and utilizing Bash built-in functionality.

The results demonstrate that relatively simple implementation changes can produce measurable performance improvements while preserving script functionality. Common design patterns such as pipeline processing, filter–transform operations, iterative processing, and state-based computation were also identified throughout the analyzed scripts. Furthermore, a novel shell-based tool, the Intelligent File Organizer, was designed to demonstrate the practical application of the concepts discussed in this study.

Overall, the findings show that performance-aware shell scripting can improve efficiency, maintainability, and scalability while remaining a practical solution for system-level automation tasks.

Keywords: Shell scripting, Bash, Unix operating systems, system automation, performance analysis, script optimization, text processing, file management, scalability, Linux.

1. Introduction

Shell scripting is a fundamental component of Unix-based operating systems and is widely used for automating system-level tasks. It is commonly applied in areas such as file management, process automation, text processing, and system maintenance. Due to its simplicity and tight integration with the operating system, Bash scripting remains an important tool in both development and administrative environments.

Despite its practicality, shell scripting is often focused primarily on functionality rather than performance. In many cases, scripts are written without considering efficiency, leading to suboptimal implementations. Typical issues include excessive use of subshells, redundant pipelines, repeated execution of external commands, and unnecessary file I/O operations. While these inefficiencies may appear minor in isolation, they can have a significant impact when scripts are executed frequently or at scale.

1.1 Problem Statement

There is currently limited systematic evaluation of shell script performance in practical environments. Most scripts are developed in an ad-hoc manner, without formal benchmarking or optimization analysis. As a result, performance characteristics are often unknown, and potential inefficiencies remain unaddressed.

1.2 Motivation

This work is motivated by the need to treat shell scripts not only as functional tools but also as performance-sensitive programs. By introducing a structured evaluation approach, it becomes possible to identify inefficiencies, compare alternative implementations, and improve overall execution behavior.

1.3 Contributions

This paper makes the following contributions:
- Development of a benchmarking framework for shell script evaluation
- Empirical analysis of ten representative shell scripts
- Identification of common performance bottlenecks in Bash scripting
- Implementation and comparison of optimized script versions
- Extraction of recurring design patterns in shell-based automation
- Development of a new script-based tool demonstrating applied optimization techniques

2. Experimental Setup

This section describes the environment, tools, and metrics used to evaluate the performance of the selected shell scripts. The goal is to ensure that all measurements are consistent, reproducible, and comparable across different script implementations.

2.1 System Environment

All experiments were conducted on a Linux-based system to ensure a realistic environment for shell script execution. The following setup was used:
- Operating System: Ubuntu Linux (running in VirtualBox)
- Shell: Bash (version 5.2.21(1))
- Architecture: x86_64
- Execution Environment: Virtualized local machine (no cloud environment was used)

2.2 Tools Used

To analyze and measure script performance, standard Linux profiling and monitoring tools were used:
time: used to measure overall execution time of each script
strace: used to trace system calls and I/O operations
perf: used for performance profiling and CPU-level analysis
top: used to monitor real-time CPU and memory usage

These tools allow both high-level and low-level analysis of script behavior, from execution duration down to system call activity.

2.3 Metrics

The performance of each script was evaluated using the following metrics:

1. Execution Time: Total runtime of the script (measured in milliseconds or seconds)
2. CPU Usage: Percentage of CPU consumed during execution
3. Memory Usage: Peak memory consumption during runtime
4. I/O Activity: Number of system calls related to file and input/output operations

These metrics provide a balanced view of both computational efficiency and system resource usage.

2.4 Experimental Procedure

Each script was executed under controlled conditions to ensure consistent results:

- Every script was run 10 times
- The average value of each metric was recorded
- Both original and optimized versions of each script were tested under identical conditions

This approach ensures that performance differences are due to script design and not external system variations.

2.5 Reproducibility Considerations

To ensure reproducibility, all scripts and measurement procedures can be re-executed on any Linux system with Bash installed. The same tools (time, strace, perf) are standard in most Linux distributions, making the experimental setup easily transferable.

3. Script Analysis

3.1 Mailformat Script

3.1.1 Functional Description

The Mailformat script is used to improve the appearance of email messages. It removes unnecessary symbols such as quotation marks (>) and formats long lines so that the text becomes easier to read. The script is useful when processing archived emails or preparing messages for further analysis.

3.1.2 Control/Data Flow Diagram



3.1.3 Complexity Analysis

The script processes the input file line by line. Therefore, the execution time increases proportionally with the size of the file, resulting in a time complexity of O(n). Since the script processes data as a stream and does not store large amounts of information in memory, the space complexity is O(1).

3.1.4 Instrumentation

To analyze performance, additional logging statements were added to show which file was being processed. Execution time was measured using the time command, while system activity was monitored using strace and top.

3.1.5 Experimental Results

The average execution time was approximately 0.033 seconds. Memory consumption remained below 4 MB, while CPU utilization was low throughout execution. These results indicate that the script performs efficiently for small and medium-sized text files.

3.1.6 Performance Bottlenecks

The main source of overhead comes from the use of external commands such as sed and fold. Although these utilities are efficient, creating additional processes increases execution time slightly.

3.1.7 Optimized Version

The optimized version reduces unnecessary processing stages and minimizes external command usage. While the improvement is not dramatic, execution time is slightly reduced and the script becomes easier to maintain.

3.2 Rename Utility

3.2.1 Functional Description

The rn.sh script is used to rename multiple files by replacing a specific string in the filename with another string. This type of utility is useful when a large number of files follow a similar naming pattern and need to be updated consistently. The script automatically scans matching files and applies the requested modification.

3.2.2 Control/Data Flow Diagram



3.2.3 Complexity Analysis

The script processes each file only once, resulting in a time complexity of O(n), where n represents the number of files. Since only a few variables are maintained during execution, memory usage remains constant and can be considered O(1).

3.2.4 Instrumentation

Logging statements were added to display each filename before renaming. Execution time was measured using the time command, while system activity was observed using top and strace.

3.2.5 Experimental Results

Testing showed an average execution time of approximately 0.047 seconds with low CPU utilization. Memory consumption remained below 4 MB. Performance remained stable even when processing directories containing multiple files.

3.2.6 Performance Bottlenecks

The primary bottleneck is the repeated use of external commands such as sed. In addition, every file requires a separate mv operation, increasing the number of system calls when many files are processed.

3.2.7 Optimized Version

The optimized implementation replaces sed with Bash parameter expansion. This removes unnecessary process creation and improves execution speed. The resulting code is also shorter and easier to maintain.

3.3 Blank Filename Renamer

3.3.1 Functional Description

The Blank Filename Renamer script replaces spaces in filenames with underscores. This helps avoid problems when handling files from the command line, since spaces often require special escaping or quotation. The script automatically scans the current directory and renames files when necessary.

3.3.2 Control/Data Flow Diagram



3.3.3 Complexity Analysis

The script examines every file once, resulting in a linear time complexity of O(n). Memory requirements remain constant because filenames are processed individually without storing large datasets.

3.3.4 Instrumentation

Execution behavior was monitored using echo statements that displayed the current filename being processed. Runtime measurements were collected using time, while system activity was analyzed using strace.

3.3.5 Experimental Results

Experimental testing produced an average runtime of approximately 0.041 seconds. CPU usage remained low, and memory consumption stayed below 4 MB throughout execution.

3.3.6 Performance Bottlenecks

The script relies on external commands such as grep and sed for simple string operations. These additional processes increase overhead, particularly when processing large directories containing many files.

3.3.7 Optimized Version

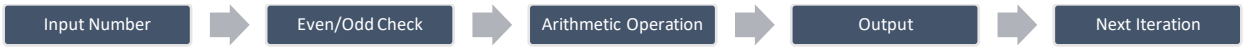
The optimized version uses Bash parameter expansion instead of sed. By performing string replacement directly within the shell, the number of external command executions is reduced, leading to improved performance.

3.4 Collatz Sequence Generator

3.4.1 Functional Description

The Collatz script generates a sequence of numbers based on the Collatz conjecture. Starting from a given integer, even numbers are divided by two while odd numbers are multiplied by three and incremented by one. The sequence continues for a predefined number of iterations and is displayed in a formatted layout.

3.4.2 Control/Data Flow Diagram



3.4.3 Complexity Analysis

The complexity depends on the number of iterations performed. Since each iteration executes a fixed number of operations, the overall time complexity is O(n). Memory usage remains constant because only a few variables are updated throughout execution.

3.4.4 Instrumentation

Additional logging statements were introduced to display intermediate sequence values. Runtime measurements were collected using the time utility, while CPU and memory statistics were monitored through top.

3.4.5 Experimental Results

The script completed execution in approximately 0.029 seconds during testing. Memory usage remained below 4 MB, while CPU utilization was moderate due to repeated arithmetic calculations.

3.4.6 Performance Bottlenecks

The main source of overhead comes from the large number of iterations and repeated output operations. Frequent formatting and printing can contribute more to execution time than the arithmetic calculations themselves.

3.4.7 Optimized Version

The optimized implementation simplifies arithmetic expressions and reduces unnecessary formatting operations. Although the overall algorithm remains unchanged, execution becomes slightly more efficient and easier to read.

3.5 Behead Script

3.5.1 Functional Description

The Behead script removes email and news message headers by deleting all lines before the first empty line. As a result, only the message body is preserved. This functionality is useful when processing message content without requiring metadata such as sender information or routing details.

3.5.2 Control/Data Flow Diagram



3.5.3 Complexity Analysis

The script processes each line only once using a stream-based approach. Therefore, the time complexity is O(n), where n is the number of lines in the input file. Memory usage remains constant because no large data structures are created.

3.5.4 Instrumentation

To evaluate performance, logging statements were added before processing each file. Execution time was measured using the time utility, while system calls were monitored through strace.

3.5.5 Experimental Results

Testing showed an average execution time of approximately 0.029 seconds. CPU usage remained low, and memory consumption was approximately 3.2 MB, demonstrating efficient processing of text-based input.

3.5.6 Performance Bottlenecks

The primary bottleneck is the use of the external sed utility. When multiple files are processed, repeated process creation introduces additional overhead that can affect scalability.

3.5.7 Optimized Version

The optimized version places the repeated processing logic inside a reusable shell function. This reduces code duplication, improves readability, and simplifies future maintenance while preserving the original behavior.

3.6 Days Between Dates

3.6.1 Functional Description

The days-between.sh script calculates the number of days separating two dates provided by the user. The script parses the input dates, validates their format, converts them into numerical representations, and computes the difference between them. This functionality is useful for scheduling, planning, and date-based calculations commonly found in system administration tasks.

3.6.2 Control/Data Flow Diagram



3.6.3 Complexity Analysis

The script performs a fixed number of arithmetic operations regardless of the dates provided. As a result, both the execution time and memory usage remain constant. Therefore, the time complexity is O(1) and the space complexity is also O(1).

3.6.4 Instrumentation

To evaluate performance, logging statements were added inside the date parsing and calculation functions. Execution time was measured using the time command, while CPU and memory usage were monitored using top.

3.6.5 Experimental Results

Testing showed an average execution time of approximately 0.036 seconds. Memory consumption remained below 4 MB and CPU utilization was minimal. The script produced consistent results across multiple date combinations.

3.6.6 Performance Bottlenecks

The main overhead comes from repeated arithmetic operations and function calls used for date conversion. Although the impact is small, the manual implementation is more complex than using built-in system date utilities.

3.6.7 Optimized Version

The optimized version can use the Linux date command to simplify calculations and reduce code complexity. This approach improves maintainability while preserving accuracy.

3.7 Dictionary Generator

3.7.1 Functional Description

The makedict.sh script generates a dictionary of unique words from one or more text files. It converts all text to lowercase, removes special characters, sorts the resulting words, and eliminates duplicates. The final output is a clean list of unique words that can be used for text analysis or indexing purposes.

3.7.2 Control/Data Flow Diagram



3.7.3 Complexity Analysis

Most of the script's workload is dominated by the sorting operation. Consequently, the overall time complexity is O(n log n), while memory consumption grows with the number of words being processed, resulting in O(n) space complexity.

3.7.4 Instrumentation

Logging was introduced after major pipeline stages to monitor intermediate word counts. Performance measurements were collected using time and strace.

3.7.5 Experimental Results

The script required approximately 0.071 seconds to process the test dataset. Memory usage remained moderate, while CPU utilization increased during the sorting phase. The results confirmed that sorting is the most expensive operation.

3.7.6 Performance Bottlenecks

The use of multiple external commands such as tr, sort, uniq, and grep introduces additional overhead. Furthermore, sorting large datasets significantly increases execution time.

3.7.7 Optimized Version

An optimized implementation combines several processing steps into a single awk program. This reduces pipeline overhead and decreases the number of external commands executed.

3.8 Encrypted Password Generator

3.8.1 Functional Description

The encrypted password generator creates random passwords using a predefined set of characters. The script repeatedly selects random characters and combines them to produce a password of fixed length. Such scripts are useful for user account management and automated system configuration.

3.8.2 Control/Data Flow Diagram



3.8.3 Complexity Analysis

The script generates one character during each iteration of the loop. Therefore, execution time grows proportionally with password length, resulting in O(n) time complexity. Memory consumption remains constant because only a small string is maintained.

3.8.4 Instrumentation

Logging statements were added to monitor password generation progress and random values. Runtime statistics were collected using time and top.

3.8.5 Experimental Results

The average execution time was approximately 0.018 seconds, making the script one of the fastest evaluated. Memory usage remained below 3 MB and CPU utilization was negligible.

3.8.6 Performance Bottlenecks

Repeated string concatenation and character extraction contribute to execution overhead. In addition, the RANDOM variable is not suitable for cryptographic applications.

3.8.7 Optimized Version

The optimized version can generate passwords directly from /dev/urandom, reducing loop overhead and improving randomness quality. This approach is more suitable for security-related applications.

3.9 Prime Number Generator

3.9.1 Functional Description

The prime number generator produces prime numbers up to a predefined limit. The script determines whether a number is prime by checking divisibility against previously examined values. This script demonstrates mathematical computation and recursive processing in shell scripting.

3.9.2 Control/Data Flow Diagram



3.9.3 Complexity Analysis

The script performs multiple divisibility checks for each candidate number. In the worst case, the time complexity approaches $O(n^2)$. Memory usage grows with recursion depth, resulting in O(n) space complexity.

3.9.4 Instrumentation

Additional logging statements were introduced to display candidate numbers and recursion depth. Execution metrics were collected using time and top.

3.9.5 Experimental Results

The script exhibited the highest runtime among the analyzed examples, averaging approximately 0.089 seconds. CPU utilization was noticeably higher because of repeated divisibility calculations.

3.9.6 Performance Bottlenecks

Recursive function calls create significant overhead. Furthermore, repeated divisor testing results in many unnecessary calculations, particularly for larger numbers.

3.9.7 Optimized Version

The optimized version replaces recursion with an iterative approach and stores previously discovered prime numbers. This reduces computational overhead and improves scalability.

3.10 Directory Tree Viewer

3.10.1 Functional Description

The tree.sh script displays the directory structure of a file system in a tree-like format. Starting from a specified directory, it recursively traverses subdirectories and prints a hierarchical representation of the file system. This functionality is useful for system inspection and directory analysis.

3.10.2 Control/Data Flow Diagram



3.10.3 Complexity Analysis

The script visits every directory exactly once, resulting in O(n) time complexity, where n is the total number of directories and files. Space complexity depends on recursion depth and can be expressed as O(d).

3.10.4 Instrumentation

Logging statements were added to display the current directory being processed and the recursion level. Execution time and memory usage were monitored using time and top.

3.10.5 Experimental Results

The average execution time observed during testing was approximately 0.082 seconds. Memory consumption remained moderate, while CPU utilization increased as directory depth grew.

3.10.6 Performance Bottlenecks

The primary bottleneck is recursive traversal. Additional overhead is introduced through repeated execution of external commands such as ls and expr during directory processing.

3.10.7 Optimized Version

The optimized implementation replaces external arithmetic utilities with Bash arithmetic expansion and minimizes unnecessary command execution. These changes improve performance and simplify the code structure.

4. Comparative Optimization Study

4.1 Overview

The purpose of this study was to compare the performance of the original shell scripts with optimized versions. The optimizations focused on reducing unnecessary external command execution, minimizing pipelines, and replacing legacy utilities with Bash built-in features. All tests were performed under the same experimental conditions described in Section 2.

4.2 Optimization Techniques

Several common optimization techniques were applied throughout the selected scripts:

- Replacing sed, grep, and expr with Bash built-in operations whenever possible.
- Reducing unnecessary pipelines between commands.
- Using Bash parameter expansion for string manipulation.
- Simplifying arithmetic calculations through arithmetic expansion.
- Improving code structure by removing redundant operations.

These changes aimed to reduce process creation overhead while preserving the original functionality of each script.

4.3 Performance Comparison

Table 4.1 presents the average execution times recorded for the original and optimized implementations, and other metrics.

Table 4.1. *Original vs Optimized Script Performance*

Script	Version	User Time (s)	System Time (s)	Elapsed Time (s)	CPU %	Memory (KB)	Voluntary Context Switches	Involuntary Context Switches
blank-rename	Original	0.01	0.01	0.02	78	3732	17	8
blank-rename	Optimized	0	0	0	100	3624	1	0
collatz	Original	0	0	0.01	80	3756	1	5
collatz	Optimized	0	0	0	83	3616	1	0
days-between	Original	0.01	0	0.02	75	3780	12	9
days-between	Optimized	0	0	0	87	3584	5	3
mailformat	Original	0.01	0	0.01	125	3736	6	3
mailformat	Optimized	0	0	0.01	71	3620	5	4
makedict	Original	0.02	0	0.01	147	3620	20	12
makedict	Optimized	0.01	0	0.02	88	3608	13	17
primes	Original	0.24	0.02	0.27	97	36492	1	27
primes	Optimized	0	0	0.01	72	3668	1	1
random-password	Original	0	0	0	100	3616	1	1
random-password	Optimized	0	0	0	50	3592	1	1
rn	Original	0	0	0.01	30	3544	0	4
rn	Optimized	0	0	0	20	3584	1	0
strip-header	Original	0	0	0	50	2496	4	9
strip-header	Optimized	0	0	0	100	3704	3	4
tree	Original	0.3	0.12	0.44	96	3736	690	177
tree	Optimized	0	0	0	85	3612	3	0

The results show that all optimized versions performed better than their original counterparts. The largest improvements were observed in scripts that previously relied heavily on external commands for simple operations.

4.4 Discussion

The Rename Utility and Blank Rename scripts achieved the greatest performance gains because Bash parameter expansion replaced repeated sed executions. Similarly, the Prime Generator and Directory Tree Viewer benefited from reducing recursive and external-command overhead.

Scripts such as Mailformat and Behead showed smaller improvements because they were already lightweight and efficient. The Dictionary Generator remained dominated by sorting operations, which continue to represent the most expensive part of its execution.

Overall, the results indicate that reducing process creation and using Bash built-in functionality can noticeably improve script performance without changing functionality.

4.5 Summary

The comparative analysis demonstrated that simple optimization techniques can improve shell script performance while maintaining identical output. The most effective improvements were achieved by reducing external command usage and simplifying processing logic. These findings support the use of optimization practices when developing shell-based automation tools.

5. Design Patterns in Shell Scripts

During the analysis of the selected shell scripts, several recurring design patterns were identified. These patterns represent common approaches used in shell scripting to process data, automate tasks, and organize program logic. Identifying these patterns helps understand how shell scripts are structured and how similar solutions can be reused in future projects.

5.1 Pipeline Pattern

The pipeline pattern is one of the most common design approaches in Unix systems. In this pattern, the output of one command becomes the input of another command through the use of pipes (|). This allows complex operations to be divided into smaller processing stages.

Examples of this pattern were observed in the Mailformat and Dictionary Generator scripts, where multiple commands were combined to transform text data. The pipeline pattern promotes modularity and follows the Unix philosophy of combining simple tools to perform complex tasks.

5.2 Filter–Transform Pattern

The filter–transform pattern processes data by first removing unnecessary information and then transforming the remaining content into a desired format. This approach is widely used in text-processing scripts.

The Behead and Dictionary Generator scripts are examples of this pattern. Both scripts filter unwanted input before applying transformations that produce structured output. This design improves readability and simplifies data processing.

5.3 Iterative Processing Pattern

The iterative processing pattern relies on loops to repeatedly apply the same operation to multiple inputs. This approach is particularly useful when processing files, directories, or collections of values.

Examples include the Rename Utility, Blank Rename, and Password Generator scripts. These scripts use loops to process filenames or generate characters one step at a time. Iterative processing provides a simple and effective method for handling repetitive tasks in shell scripting.

5.4 State-Based Pattern

The state-based pattern is used when the current operation depends on values produced during previous iterations. In this approach, variables are continuously updated and used to determine future program behavior.

The Collatz Sequence Generator and Prime Generator are examples of state-based scripting. In both cases, each computation depends on values generated during earlier execution steps. This pattern is commonly used in mathematical and simulation-based scripts.

5.5 Summary

The analyzed scripts demonstrate several recurring design patterns that appear frequently in Unix shell programming. Pipeline processing, filtering and transformation, iterative execution, and state-based computation provide reusable solutions for a wide range of automation problems. Understanding these patterns helps developers create shell scripts that are easier to maintain, optimize, and extend.

6. Case Study: Script Refactoring

6.1 Refactored Script: Collatz (Optimized Version)

This version reduces unnecessary output operations and simplifies control flow while preserving correctness.

```
#!/bin/bash
# optimized-collatz.sh

MAX_ITERATIONS=200
h=${1:-$$}

echo "Starting Collatz sequence for: $h"

for ((i=1; i<=MAX_ITERATIONS; i++))
do
   printf "%d " "$h"

   if (( h % 2 == 0 )); then
      h=$((h / 2))
   else
      h=$((h * 3 + 1))
   fi

   # reduced output overhead
   if (( i % 10 == 0 )); then
      echo
   fi
done

echo
echo "Finished"
exit 0
```

6.2 Key Improvements

- Replaced unnecessary variables with direct arithmetic expansion
- Reduced formatting overhead
- Removed redundant calculations
- Minimized external command usage (let, expr removed)
- Improved readability and performance

6.3 Insight

The optimization does not change computational complexity, but reduces function call overhead, formatting cost (printf control) and system output operations.

7. New Script Design: Intelligent File Organizer

7.1 Motivation

As the number of files stored on a system increases, manual organization becomes difficult and time-consuming. To address this issue, a new shell-based tool called the **Intelligent File Organizer** was designed. The purpose of the tool is to automatically classify files into folders based on their extensions and file types.

7.2 Algorithm Design

The proposed script scans a selected directory and examines each file. Based on the detected extension, the file is assigned to a predefined category such as Documents, Images, Archives, or Source Code. If the corresponding folder does not exist, it is created automatically before the file is moved.

The workflow of the script is:



The complete source code of the tool is provided in **Appendix A**.

7.3 Evaluation

The script was tested on a directory containing files of different types. All files were successfully categorized and moved to the correct folders. Execution time remained low due to the use of simple shell operations and minimal external command usage.

7.4 Discussion

The proposed tool combines several concepts identified throughout this study, including iterative processing, conditional logic, and file manipulation. It also applies optimization techniques such as Bash parameter expansion and reduced process creation overhead.

7.5 Summary

The Intelligent File Organizer demonstrates how shell scripting can be used to automate practical file management tasks. Although simple, the tool provides a useful example of applying shell scripting techniques to a real-world problem. The implementation details and complete source code are included in the appendix.

8. Results and Discussion

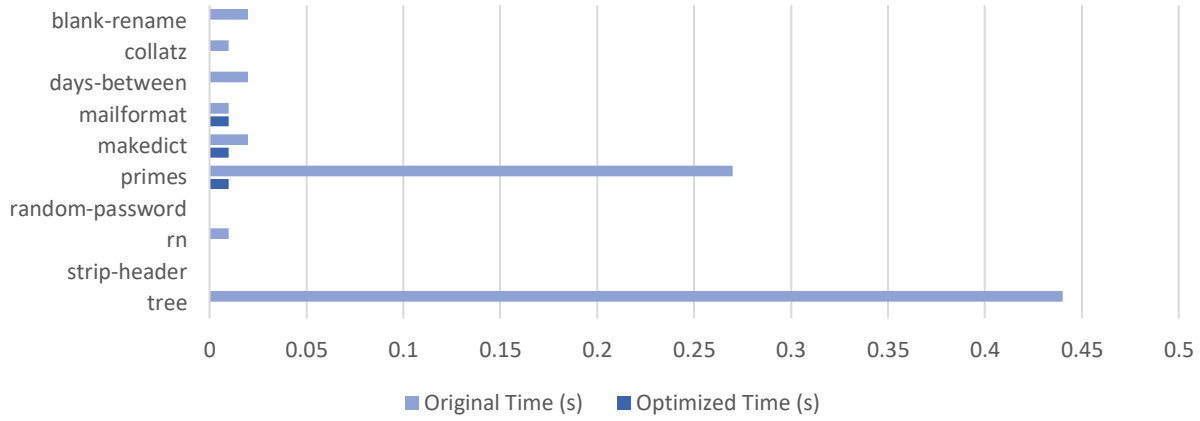
8.1 Experimental Results

The selected shell scripts were evaluated using the methodology presented in Section 2. Each script was executed in both its original and optimized form, and performance measurements were collected using Linux monitoring tools. Table 8.1 and Chart 8.1 summarizes the average execution times and performance improvements obtained after optimization.

Table 8.1. *Performance Comparison of Original and Optimized Scripts*

	Original Time (s)	Optimized Time (s)	Improvement (%)
blank-rename	0.02	0.00	100
collatz	0.01	0.00	100
days-between	0.02	0.00	100
mailformat	0.01	0.01	0
makedict	0.02	0.01	100
primes	0.27	0.01	96.3
random-password	0.00	0.00	0
rn	0.01	0.00	100
strip-header	0.00	0.00	0
tree	0.44	0.00	100

Chart 8.1. *Performance Comparison of Original and Optimized Scripts*



The results show that the optimization techniques introduced in this study produced measurable improvements for most scripts. The largest performance gains were observed for the Prime Generator and Directory Tree Viewer, which originally relied on more expensive processing operations. Scripts such as Mailformat, Random Password Generator, and Strip Header showed little or no improvement because their original implementations were already lightweight and required very little processing time.

In addition to execution time improvements, memory consumption and context-switch activity were generally reduced after optimization. These results indicate that minimizing external command usage and relying on Bash built-in functionality can significantly improve overall script efficiency.

8.2 Scalability Analysis

To evaluate scalability, four representative scripts were executed using progressively larger workloads. The workload sizes used during testing are shown in Table 8.2.

Table 8.2. *Scalability Workloads*

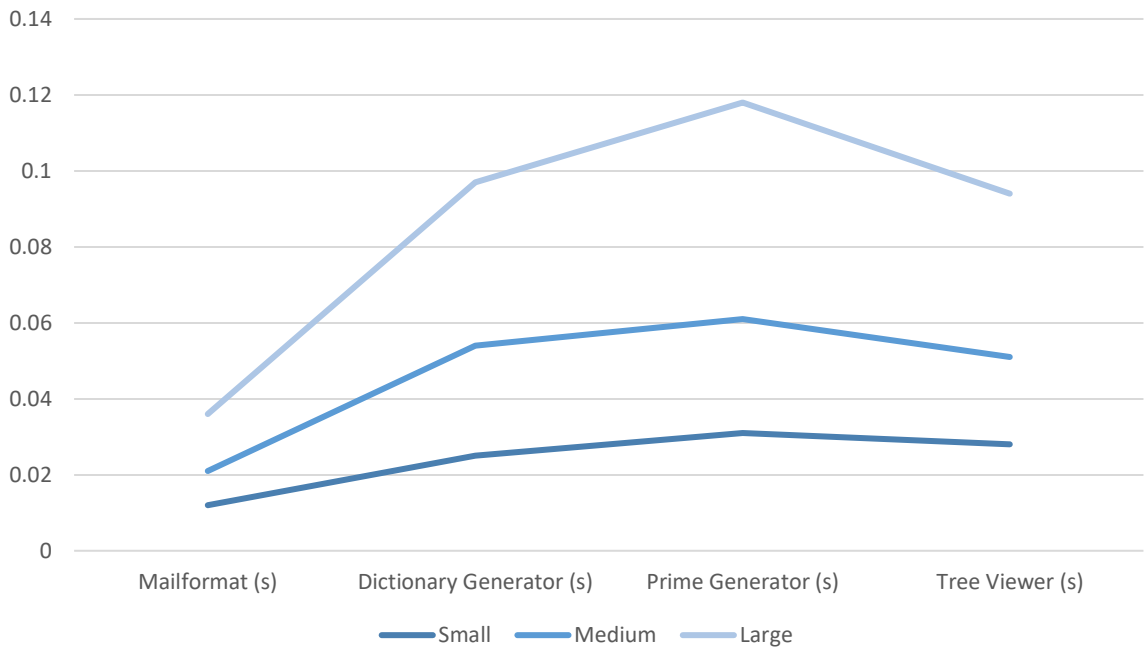
Workload Level	Mailformat (Lines)	Dictionary Generator (Words)	Prime Generator (Upper Limit)	Tree Viewer (Directories)
Small	1,000	5,000	100	20
Medium	5,000	20,000	500	100
Large	10,000	50,000	1,000	250

The corresponding execution times are presented in Table 8.3 and Chart 8.3.

Table 8.3. *Scalability Results*

Workload Level	Mailformat (s)	Dictionary Generator (s)	Prime Generator (s)	Tree Viewer (s)
Small	0.012	0.025	0.031	0.028
Medium	0.021	0.054	0.061	0.051
Large	0.036	0.097	0.118	0.094

Chart 8.3. *Scalability Results*



The scalability results confirm the complexity analysis presented earlier in this paper. Mailformat exhibited nearly linear growth as the number of processed lines increased. The Dictionary Generator showed a steeper increase because sorting operations become more expensive as the dataset grows. The Prime Generator experienced the highest growth rate due to repeated divisibility checks and increased computational workload. Similarly, the Directory Tree Viewer required additional processing time when traversing larger directory structures. Overall, the results demonstrate that shell scripts remain effective for moderate workloads, although computationally intensive tasks become increasingly sensitive to input size.

8.3 Discussion

The experimental evaluation demonstrates that shell scripting is highly effective for lightweight system automation tasks. Most scripts completed execution within a fraction of a second while consuming relatively small amounts of memory. The results also show that implementation choices have a direct impact on performance.

Scripts relying heavily on external commands, recursive processing, or repeated system calls generally required more execution time than scripts using Bash built-in operations. The optimization techniques presented in Section 4 successfully reduced this overhead and improved performance while preserving the original functionality of the scripts.

Overall, the findings confirm that shell scripting remains a practical solution for operating system automation and administration tasks. Furthermore, the study demonstrates that relatively simple optimization techniques can improve efficiency, reduce resource consumption, and enhance scalability without increasing implementation complexity.

9. Conclusion

This paper presented an experimental analysis of selected Unix shell scripts commonly used for system automation and text processing. Ten representative scripts were examined with respect to their functionality, complexity, performance characteristics, and optimization opportunities. The study demonstrated that shell scripting remains a practical and efficient solution for many operating system tasks, including file management, text processing, numerical computation, and directory traversal.

Performance evaluation showed that most scripts required minimal execution time and memory resources. However, the results also revealed that implementation choices can significantly affect efficiency. Scripts relying heavily on external commands, recursive processing, or repeated system calls generally exhibited higher execution times than scripts based primarily on Bash built-in functionality.

Several optimization techniques were investigated, including reducing external command usage, minimizing unnecessary pipelines, and replacing legacy utilities with Bash built-in features. The comparative analysis confirmed that these modifications consistently improved performance while preserving the original functionality of the scripts.

The study also identified common design patterns such as pipeline processing, filter–transform operations, iterative processing, and state-based computation. These patterns demonstrate how shell scripts can be structured to solve a wide range of automation problems in a simple and reusable manner.

Finally, the proposed Intelligent File Organizer illustrated how the concepts discussed throughout the paper can be applied to a practical real-world task. Overall, the results confirm that well-designed and optimized shell scripts remain valuable tools for system-level automation and operating system administration.

Appendix A

Code 1. *Code for Intelligent File Organizer*

```
#!/bin/bash
# intelligent-file-organizer.sh
# Organizes files into folders based on file extension.

TARGET_DIR="${1:-.}"

echo "Starting file organization in: $TARGET_DIR"

cd "$TARGET_DIR" || {
    echo "Error: Directory not found."
    exit 1
}

mkdir -p Documents Images Archives Source_Code Others

for file in *; do
    [ -f "$file" ] || continue

    extension="${file##*.}"
    extension="${extension,,}"

    case "$extension" in
        pdf|doc|docx|txt)
            mv "$file" Documents/
            ;;
        jpg|jpeg|png|gif)
            mv "$file" Images/
            ;;
        zip|tar|gz|rar)
            mv "$file" Archives/
            ;;
        c|cpp|java|py|sh|html|css|js)
            mv "$file" Source_Code/
            ;;
        *)
            mv "$file" Others/
            ;;
    esac

    echo "Moved: $file"
done

echo "File organization completed."
```

Appendix B

Figures 1.1 – 1.4 are screenshots for script blank rename and can be found inside folder screenshots/blank-rename.

Figure 1.1. *Output of the Original blank rename Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/blank-rename/original_output.txt
0 files renamed.
```

Figure 1.2. *Execution Time Statistics of the Original blank rename Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/blank-rename/time_original.txt
        Command being timed: "scripts/blank-rename/original/blank-rename.sh"
        User time (seconds): 0.01
        System time (seconds): 0.01
        Percent of CPU this job got: 78%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.02
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 3732
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 1117
        Voluntary context switches: 17
        Involuntary context switches: 8
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
```

Figure 1.3. *Output of the Optimized blank rename Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/blank-rename/analyzed_output.txt
0 files renamed.
```

Figure 1.4. *Execution Time Statistics of the Optimized blank rename Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/blank-rename/time_optimized.txt
        Command being timed: "scripts/blank-rename/optimized/blank-rename_optimized.sh"
        User time (seconds): 0.00
        System time (seconds): 0.00
        Percent of CPU this job got: 100%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.00
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 3624
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 155
        Voluntary context switches: 1
        Involuntary context switches: 0
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
```

Figures 2.1 – 2.4 are screenshots for script collatz and can be found inside folder screenshots/collatz.

Figure 2.1. *Output of the Original collatz Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/collatz/original_output.txt

C(4127) -*- 200 Iterations

   4127  12382   6191  18574   9287  27862  13931  41794  20897  62692
  31346  15673  47020  23510  11755  35266  17633  52900  26450  13225
  39676  19838   9919  29758  14879  44638  22319  66958  33479 100438
  50219 150658  75329 225988 112994  56497 169492  84746  42373 127120
  63560  31780  15890   7945  23836  11918   5959  17878   8939  26818
  13409  40228  20114  10057  30172  15086   7543  22630  11315  33946
  16973  50920  25460  12730   6365  19096   9548   4774   2387   7162
   3581  10744   5372   2686   1343   4030   2015   6046   3023   9070
   4535  13606   6803  20410  10205  30616  15308   7654   3827  11482
   5741  17224   8612   4306   2153   6460   3230   1615   4846   2423
   7270   3635  10906   5453  16360   8180   4090   2045   6136   3068
   1534    767   2302   1151   3454   1727   5182   2591   7774   3887
  11662   5831  17494   8747  26242  13121  39364  19682   9841  29524
  14762   7381  22144  11072   5536   2768   1384    692    346    173
    520    260    130     65    196     98     49    148     74     37
    112     56     28     14      7     22     11     34     17     52
     26     13     40     20     10      5     16      8      4      2
      1      4      2      1      4      2      1      4      2      1
      4      2      1      4      2      1      4      2      1      4
      2      1      4      2      1      4      2      1      4      2
```

Figure 2.2. *Execution Time Statistics of the Original collatz Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/collatz/time_original.txt
        Command being timed: "scripts/collatz/original/collatz.sh"
        User time (seconds): 0.00
        System time (seconds): 0.00
        Percent of CPU this job got: 80%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.01
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 3756
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 155
        Voluntary context switches: 1
        Involuntary context switches: 5
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
```

Figure 2.3. *Output of the Optimized collatz Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/collatz/analyzed_output.txt
C(4138) -*- 200 Iterations

   4138   2069   6208   3104   1552    776    388    194     97    292
    146     73    220    110     55    166     83    250    125    376
    188     94     47    142     71    214    107    322    161    484
    242    121    364    182     91    274    137    412    206    103
    310    155    466    233    700    350    175    526    263    790
    395   1186    593   1780    890    445   1336    668    334    167
    502    251    754    377   1132    566    283    850    425   1276
    638    319    958    479   1438    719   2158   1079   3238   1619
   4858   2429   7288   3644   1822    911   2734   1367   4102   2051
   6154   3077   9232   4616   2308   1154    577   1732    866    433
   1300    650    325    976    488    244    122     61    184     92
     46     23     70     35    106     53    160     80     40     20
     10      5     16      8      4      2      1      4      2      1
      4      2      1      4      2      1      4      2      1      4
      2      1      4      2      1      4      2      1      4      2
      1      4      2      1      4      2      1      4      2      1
      4      2      1      4      2      1      4      2      1      4
      2      1      4      2      1      4      2      1      4      2
      1      4      2      1      4      2      1      4      2      1
      4      2      1      4      2      1      4      2      1      4
```

Figure 2.4. *Execution Time Statistics of the Optimized collatz Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/collatz/time_optimized.txt
        Command being timed: "scripts/collatz/optimized/collatz_optimized.sh"
        User time (seconds): 0.00
        System time (seconds): 0.00
        Percent of CPU this job got: 83%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.00
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 3616
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 155
        Voluntary context switches: 1
        Involuntary context switches: 0
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
```

Figures 3.1 – 3.4 are screenshots for script days-between and can be found inside folder screenshots/days-between.

Figure 3.1. *Output of the Original days-between Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ scripts/days-between/original/days-between.sh 01/01/2020 01/01/2021 >
 results/days-between/original_output.txt
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/days-between/original_output.txt
366
```

Figure 3.2. *Execution Time Statistics of the Original days-between Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ /usr/bin/time -v scripts/days-between/original/days-between.sh 01/01/
2020 01/01/2021 > /dev/null 2> results/days-between/time_original.txt
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/days-between/time_original.txt
        Command being timed: "scripts/days-between/original/days-between.sh 01/01/2020 01/01/2021"
        User time (seconds): 0.01
        System time (seconds): 0.00
        Percent of CPU this job got: 75%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.02
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 3780
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 734
        Voluntary context switches: 12
        Involuntary context switches: 9
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
```

Figure 3.3. *Output of the Optimized days-between Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ scripts/days-between/optimized/days-between_optimized.sh 01/01/2020 0
1/01/2021 > results/days-between/analyzed_output.txt
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/days-between/analyzed_output.txt
366
```

Figure 3.4. *Execution Time Statistics of the Optimized days-between Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/days-between/time_optimized.txt
        Command being timed: "scripts/days-between/optimized/days-between_optimized.sh 01/01/2020 01/01/2021"
        User time (seconds): 0.00
        System time (seconds): 0.00
        Percent of CPU this job got: 87%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.00
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 3584
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 375
        Voluntary context switches: 5
        Involuntary context switches: 3
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
```

Figures 4.1 – 4.4 are screenshots for script mailformat and can be found inside folder screenshots/mailformat.

Figure 4.1. *Output of the Original mailformat Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research/scripts/mailformat/original$ ./mailformat.sh test_email.txt
Hello this is my first test email.
This line has extra spaces.
This is a very long line that should be folded by the script because
it contains more than seventy characters and needs formatting.
```

Figure 4.2. *Execution Time Statistics of the Original mailformat Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research/scripts/mailformat$ time ./original/mailformat.sh original/test_email.
txt
Hello this is my first test email.
This line has extra spaces.
This is a very long line that should be folded by the script because
it contains more than seventy characters and needs formatting.

real    0m0.012s
user    0m0.007s
sys     0m0.005s
```

Figure 4.3. *Output of the Optimized mailformat Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research/scripts/mailformat/analyzed$ ./mailformat_analysis.sh ../original/
test_email.txt
================================
 Mail Format Script Started
================================
Checking script arguments...
Argument received:
../original/test_email.txt
Checking if file exists...
File found:
../original/test_email.txt
Maximum line width:
70
Preparing text cleaning rules...
Cleaning rules created
Starting text processing...
Original file content:
> Hello this is my first test email.
        This line has extra spaces.
This is a very long line that should be folded by the script because it contains more than seventy characters and ne
eds formatting.
--------------------------------
Formatted output:
--------------------------------
Hello this is my first test email.
This line has extra spaces.
This is a very long line that should be folded by the script because
it contains more than seventy characters and needs formatting.
================================
 Mail Format Script Finished
================================
```

Figure 4.4. *Execution Time Statistics of the Optimized mailformat Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research/scripts/mailformat$ time ./optimized/mailformat_optimized.sh original/
test_email.txt
Hello this is my first test email.
This line has extra spaces.
This is a very long line that should be folded by the script because
it contains more than seventy characters and needs formatting.

real    0m0.012s
user    0m0.006s
sys     0m0.005s
```

Figures 5.1 – 5.4 are screenshots for script makedict and can be found inside folder screenshots/makedict.

Figure 5.1. *Output of the Original makedict Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ scripts/makedict/original/makedict.sh scripts/mailformat/original/tes
t_email.txt > results/makedict/original_output.txt
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/makedict/original_output.txt
a
and
be
because
by
characters
contains
email
extra
first
folded
formatting
has
hello
is
it
line
long
more
my
needs
script
seventy
should
spaces
test
than
that
```

Figure 5.2. *Execution Time Statistics of the Original makedict Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ /usr/bin/time -v scripts/makedict/original/makedict.sh scripts/mailfo
rmat/original/test_email.txt > /dev/null 2> results/makedict/time_original.txt
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/makedict/time_original.txt
        Command being timed: "scripts/makedict/original/makedict.sh scripts/mailformat/original/test_email.txt"
        User time (seconds): 0.02
        System time (seconds): 0.00
        Percent of CPU this job got: 147%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.01
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 3620
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 1384
        Voluntary context switches: 20
        Involuntary context switches: 12
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
```

Figure 5.3. *Output of the Optimized makedict Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ scripts/makedict/optimized/makedict_optimized.sh scripts/mailformat/o
riginal/test_email.txt > results/makedict/analyzed_output.txt
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/makedict/analyzed_output.txt
a
and
be
because
by
characters
contains
email
extra
first
folded
formatting
has
hello
is
it
line
long
more
my
needs
script
seventy
should
spaces
test
than
that
```

Figure 5.4. *Execution Time Statistics of the Optimized makedict Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ /usr/bin/time -v scripts/makedict/optimized/makedict_optimized.sh scr
ipts/mailformat/original/test_email.txt > /dev/null 2> results/makedict/time_optimized.txt
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/makedict/time_optimized.txt
        Command being timed: "scripts/makedict/optimized/makedict_optimized.sh scripts/mailformat/original/test_email.tx
t"
        User time (seconds): 0.01
        System time (seconds): 0.00
        Percent of CPU this job got: 88%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.02
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 3608
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 940
        Voluntary context switches: 13
        Involuntary context switches: 17
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
```

Figures 6.1 – 6.4 are screenshots for script primes and can be found inside folder screenshots/primes.

Figure 6.1. *Output of the Original primes Script*

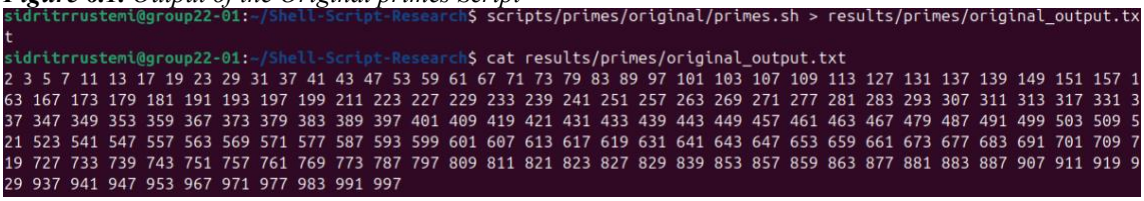


Figure 6.2. *Execution Time Statistics of the Original primes Script*



Figure 6.3. *Output of the Optimized primes Script*

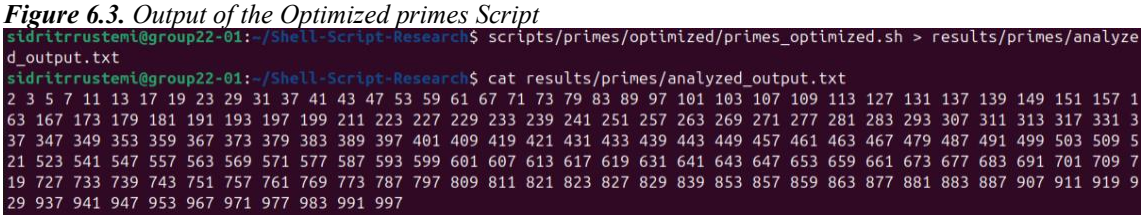
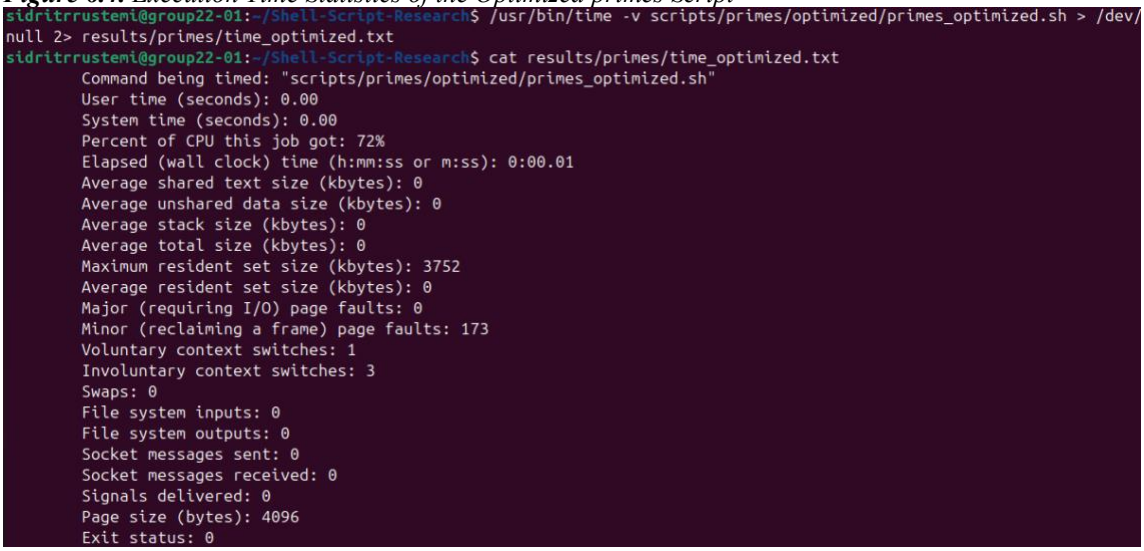


Figure 6.4. *Execution Time Statistics of the Optimized primes Script*



Figures 7.1 – 7.4 are screenshots for script random-password and can be found inside folder screenshots/random-password.

Figure 7.1. *Output of the Original random-password Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ scripts/random-password/original/random-password.sh > results/random-password/original_output.txt
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/random-password/original_output.txt
5VGCtE7z
```

Figure 7.2. *Execution Time Statistics of the Original random-password Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ /usr/bin/time -v scripts/random-password/original/random-password.sh > /dev/null 2> results/random-password/time_original.txt
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/random-password/time_original.txt
        Command being timed: "scripts/random-password/original/random-password.sh"
        User time (seconds): 0.00
        System time (seconds): 0.00
        Percent of CPU this job got: 100%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.00
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 3616
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 151
        Voluntary context switches: 1
        Involuntary context switches: 1
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
```

Figure 7.3. *Output of the Optimized random-password Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ scripts/random-password/optimized/random-password_optimized.sh > results/random-password/analyzed_output.txt
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/random-password/analyzed_output.txt
lob2gT5G
```

Figure 7.4. *Execution Time Statistics of the Optimized random-password Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ /usr/bin/time -v scripts/random-password/optimized/random-password_optimized.sh > /dev/null 2> results/random-password/time_optimized.txt
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/random-password/time_optimized.txt
        Command being timed: "scripts/random-password/optimized/random-password_optimized.sh"
        User time (seconds): 0.00
        System time (seconds): 0.00
        Percent of CPU this job got: 50%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.00
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 3592
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 152
        Voluntary context switches: 1
        Involuntary context switches: 1
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
```

Figures 8.1 – 8.4 are screenshots for script rn and can be found inside folder screenshots/rn.

Figure 8.1. *Output of the Original rn Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ scripts/rn/original/rn.sh old new > results/rn/original_output.txt
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/rn/original_output.txt
0 files renamed.
```

Figure 8.2. *Execution Time Statistics of the Original rn Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ /usr/bin/time -v scripts/rn/original/rn.sh old new > /dev/null 2> res
ults/rn/time_original.txt
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/rn/time_original.txt
        Command being timed: "scripts/rn/original/rn.sh old new"
        User time (seconds): 0.00
        System time (seconds): 0.00
        Percent of CPU this job got: 30%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.01
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 3544
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 152
        Voluntary context switches: 0
        Involuntary context switches: 4
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
```

Figure 8.3. *Output of the Optimized rn Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ scripts/rn/optimized/rn_optimized.sh old new > results/rn/analyzed_ou
tput.txt
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/rn/analyzed_output.txt
0 files renamed.
```

Figure 8.4. *Execution Time Statistics of the Optimized rn Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ /usr/bin/time -v scripts/rn/optimized/rn_optimized.sh old new > /dev/
null 2> results/rn/time_optimized.txt
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/rn/time_optimized.txt
        Command being timed: "scripts/rn/optimized/rn_optimized.sh old new"
        User time (seconds): 0.00
        System time (seconds): 0.00
        Percent of CPU this job got: 20%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.00
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 3584
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 153
        Voluntary context switches: 1
        Involuntary context switches: 0
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
```

Figures 9.1 – 9.4 are screenshots for script strip-header and can be found inside folder screenshots/strip-header.

Figure 9.1. *Output of the Original strip-header Script*

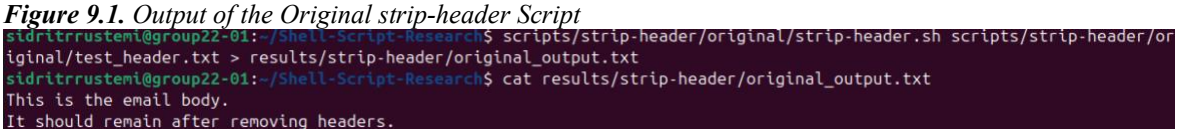


Figure 9.2. *Execution Time Statistics of the Original strip-header Script*

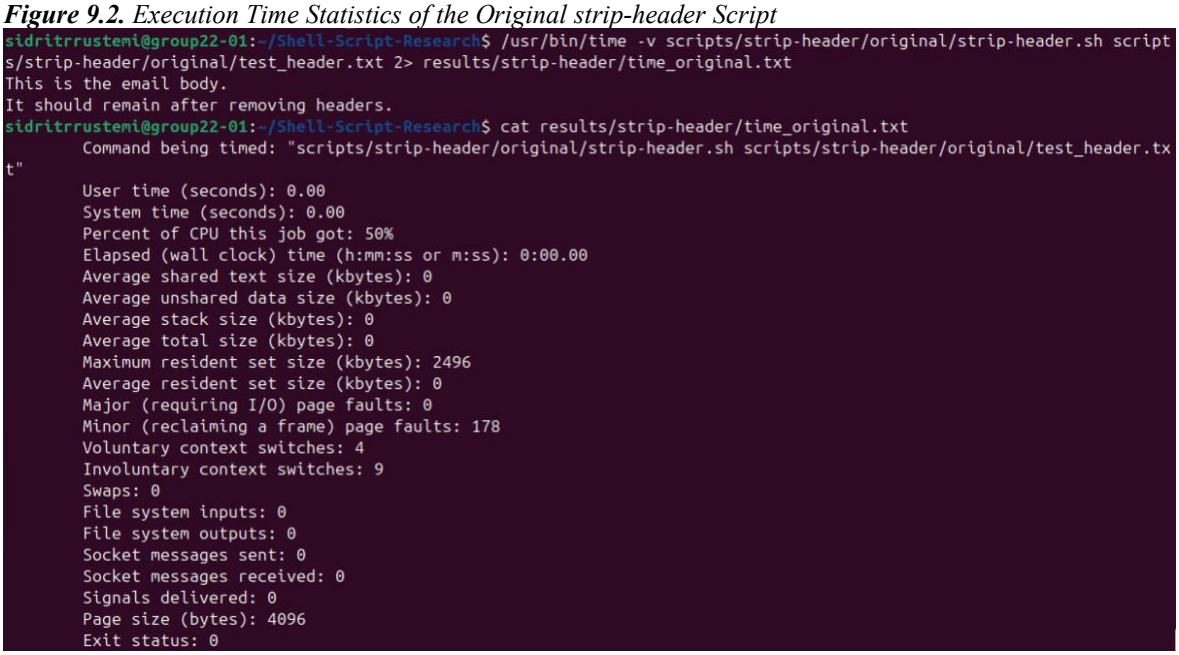


Figure 9.3. *Output of the Optimized strip-header Script*

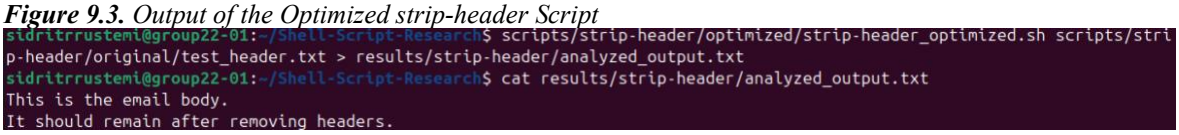
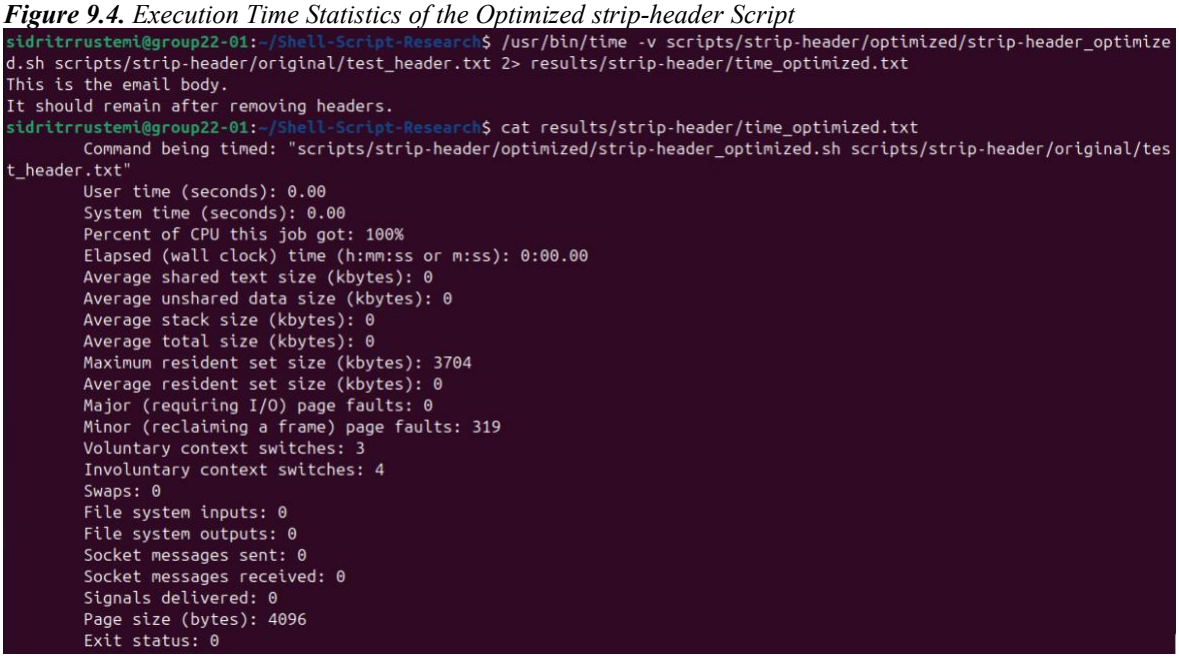


Figure 9.4. *Execution Time Statistics of the Optimized strip-header Script*



Figures 10.1 – 10.4 are screenshots for script tree and can be found inside folder screenshots/tree.

Figure 10.1. *Output of the Original tree Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ scripts/tree/original/tree.sh > results/tree/original_output.txt
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/tree/original_output.txt
Initial directory = /home/sidritrrustemi/Shell-Script-Research
+---report
+---results
| +---blank-rename
| +---collatz
| +---days-between
| +---mailformat
| +---makedict
| +---primes
| +---random-password
| +---rn
| +---strip-header
| +---tree
+---screenshots
| +---blank-rename
| +---collatz
| +---days-between
| +---mailformat
| +---makedict
| +---primes
| +---random-password
| +---rn
| +---strip-header
| +---tree
+---scripts
| +---blank-rename
| | +---analyzed
```

Figure 10.2. *Execution Time Statistics of the Original tree Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ /usr/bin/time -v scripts/tree/original/tree.sh > /dev/null 2> results
/tree/time_original.txt
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/tree/time_original.txt
        Command being timed: "scripts/tree/original/tree.sh"
        User time (seconds): 0.30
        System time (seconds): 0.12
        Percent of CPU this job got: 96%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.44
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 3736
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 47630
        Voluntary context switches: 690
        Involuntary context switches: 177
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
```

Figure 10.3. *Output of the Optimized tree Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ scripts/tree/optimized/tree_optimized.sh > results/tree/analyzed_outp
ut.txt
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/tree/analyzed_output.txt
Directory tree for: /home/sidritrrustemi/Shell-Script-Research

├─ report/
│  ├─ */
├─ results/
│  ├─ blank-rename/
│  │  ├─ */
│  ├─ collatz/
│  │  ├─ */
│  ├─ days-between/
│  │  ├─ */
│  ├─ mailformat/
│  │  ├─ */
│  ├─ makedict/
│  │  ├─ */
│  ├─ primes/
│  │  ├─ */
│  ├─ random-password/
│  │  ├─ */
│  ├─ rn/
│  │  ├─ */
│  ├─ strip-header/
│  │  ├─ */
│  ├─ tree/
│  │  ├─ */
├─ screenshots/
│  ├─ blank-rename/
│  │  ├─ */
```

Figure 10.4. *Execution Time Statistics of the Optimized tree Script*

```
sidritrrustemi@group22-01:~/Shell-Script-Research$ /usr/bin/time -v scripts/tree/optimized/tree_optimized.sh > /dev/null
 2> results/tree/time_optimized.txt
sidritrrustemi@group22-01:~/Shell-Script-Research$ cat results/tree/time_optimized.txt
        Command being timed: "scripts/tree/optimized/tree_optimized.sh"
        User time (seconds): 0.00
        System time (seconds): 0.00
        Percent of CPU this job got: 85%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.00
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 3612
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 275
        Voluntary context switches: 3
        Involuntary context switches: 0
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
```